

# Trabajo Práctico

Servidor web minimalista

Bautista Goya, Timo Doeswijk y Gonzalo Quiroga

Tecnología Digital IV: Redes de Computadoras  
Universidad Torcuato Di Tella

Sección 2 - Prof. Marcelo Romeo

2° semestre del 2025

# Introducción

## *Objetivo del trabajo práctico*

El objetivo del trabajo es implementar un servidor web minimalista que permite la transferencia de archivos entre dispositivos conectados en una misma red local. El servidor deberá permitir subir o descargar archivos según el modo seleccionado previo a la ejecución, y mostrar en consola un código QR que facilite la conexión al servidor desde otro dispositivo. El trabajo explora la capa de aplicación en el modelo TCP/IP y el funcionamiento de los sockets TCP.

Los dos modos de operación del servidor son:

- **Download:** Permite a un cliente descargar un archivo específico. Para correr el servidor en este modo se debe ejecutar el comando `python server_fileTransfer.py download {nombre archivo a descargar}`
- **Upload:** Permite a un cliente subir un archivo al servidor. Se debe ejecutar el comando `python server_fileTransfer.py upload`

## *Contexto de la capa de aplicación en el modelo TCP/IP*

En el “top-down approach” de la pila de protocolos que plantea Kurose, la llamada capa de aplicación es la más alta. En esta capa, las aplicaciones intercambian mensajes para implementar servicios y dar soporte usando servicios de la capa de transporte. Uno de los protocolos de aplicación es HTTP, usado para solicitudes y transferencia de documentos web.

Los procesos son programas en ejecución en hosts que se comunican mediante sockets. Los hosts se comunican mediante el intercambio de mensajes (Inter-Process Communication), y este intercambio de mensajes es determinado, en primer lugar, por los protocolos de la capa de aplicación.

En una arquitectura de aplicación de red de tipo cliente-servidor web (requerido en este trabajo), existe un host siempre encendido y de IP fija conocida llamado servidor, el cual responde solicitudes de objetos de otros hosts que están corriendo respectivos buscadores, los clientes. Además de direcciones IP, cada host diferencia el proceso que debe recibir ese mensaje mediante puertos numerados. Por ejemplo, el servidor web es identificado por el puerto 80 (puerto predeterminado para el protocolo HTTP).

Un socket es un conjunto de dirección IP y puerto, que opera como punto de conexión entre dos sistemas e identifica procesos. Los procesos envían y reciben mensajes a través de los sockets. Hay dos sockets involucrados en una conexión, uno en cada extremo. En el contexto de nuestro trabajo, usamos stream sockets que se implementan sobre el protocolo de transporte TCP. Estos sockets se identifican por una 4-upla que contiene dirección IP y puerto tanto origen como destino. También, el servidor cuenta con un socket de escucha mediante el cuál recibe nuevas solicitudes de conexión.

La capa de transporte se encarga de la transferencia de datos entre procesos. Un protocolo de la capa de transporte es TCP (confiable, orientado a conexión). En nuestro trabajo implementamos TCP/IP, siendo IP un protocolo del nivel de red donde se utilizan algoritmos de selección de rutas para enrutar y direccionar paquetes de datos, que puedan viajar a través de redes y llegar a su destino correcto.

La capa de aplicación en el modelo TCP/IP, es el punto donde las aplicaciones interactúan con la red. La aplicación define el contenido y el contexto, decidiendo qué tipo de datos se envían (texto, comandos, archivos, formularios), cómo se interpretan y cómo interactúa el usuario o el software.

También, algunos protocolos y medidas de seguridad importantes se aplican sobre la capa de aplicación.

### *Finalidad del servidor*

La finalidad del servidor implementado es intercambiar archivos entre distintos hosts de una misma red de manera confiable (TCP/IP da garantías contra pérdidas, duplicados, reordenamientos, datos corruptos, control de flujo y congestión), y proveyendo una interfaz web simple (bajo el protocolo HTTP), para facilitar el intercambio.

## Implementación

### *Generación del QR*

La librería `qrcode` incluye una forma de imprimir los QRs generados en la consola mediante el recurso del arte ASCII: una forma de concatenar caracteres, en este caso cuadrados y espacios vacíos, para replicar una imagen. Por esto, la aplicación de la función fue bastante sencilla.

### *Manejo de solicitud de descarga y subida*

Ya sabiendo la sintaxis de la respuesta HTTP gracias a la materia, el primer problema que nos cruzamos fue cómo construirla en Python. Aprendimos que este se envía por sockets en bytes, no en strings, por lo que se debía usar `b"..."` o `.encode()` para traducir el texto. Sabiendo esto, e indagando sobre la forma de construir las respuestas en este lenguaje, construimos los respectivos casos de error (404 Not Found para la descarga y 500 Internal Server Error para la subida) y el caso 200 OK de éxito para ambos.

También, construimos un archivo HTML para poder volver a cargar archivos cuando la carga se realiza con éxito, ya que de caso contrario el servidor seguiría corriendo pero no podría recibir solicitudes nuevas. En el mismo, al apretar volver, se vuelve a solicitar conexión y se envía un mensaje HTTP con el método GET para solicitar la página de carga de archivos.

### *Inicio del servidor*

Inicializar el servidor fue una de las partes más laboriosas del trabajo. Para la implementación de esta función, tuvimos que indagar en distintos documentos para comprender el manejo de sockets y conexiones en Python, e intentar mediante la prueba y error.

Lo primero que hicimos fue crear el socket de escucha, el cuál nos servirá para recibir solicitudes de conexiones. Para ello, obtuvimos la dirección IP de la red local y asignamos un número de puerto aleatorio para cada nueva conexión. Inicialmente optamos por establecer un número de puerto fijo, pero consideramos que era una mejor opción asignar un número aleatorio (usando la librería `random`) y así evitar cualquier posible error (cada conexión debe tener un conjunto de IP y puerto único). El número de puerto de cada conexión está entre 1024 y 8192 (por simplicidad).

Al ser un servidor web que utiliza el protocolo HTTP, el puerto asignado en el extremo del servidor debería ser el puerto 80 (predeterminado para HTTP). Sin embargo, el puerto 80 es un puerto privilegiado (menor a 1024) por lo que no puede ser asignado directamente usando `bind` en Python. Para poder asignarlo, el servidor se debe ejecutar como root, y esto no sucede en nuestro caso.

Es importante aclarar que el modelo TCP/IP utilizado en el trabajo utiliza conexiones no persistentes. Es por eso que en cada mensaje HTTP se realiza una nueva conexión y se finaliza una vez que se envía la respuesta. Cuando una conexión finaliza, el estado del servidor vuelve a ser de escucha para una nueva conexión.

El paso siguiente fue proveer el código QR y URL al cliente, usando las funciones auxiliares anteriormente descritas. Cuando el cliente abre la URL solicita la conexión, el servidor la acepta y revisa el request HTTP para verificar qué modo está solicitando. Luego de decodificar el mensaje y descartar errores, generamos la respuesta.

Para generar la respuesta acordeamente, nos basamos en el método HTTP:

- Si es un GET, hay dos opciones. Si está solicitando la página inicial para descargar/subir archivos (path = "/"), enviamos el archivo HTML correspondiente. Por otro lado, si solicita el archivo a descargar (path = "/download"), el contenido de la respuesta será el archivo. En ambos casos, enviamos los archivos con la respuesta 200 OK.
- Si el método es POST, se trata de la subida de un archivo. Por lo tanto, decodificamos el mensaje guardando el archivo y enviamos la respuesta exitosa.
- Si en cualquiera de estos casos ocurre un error, respondemos con el mensaje 405 Method Not Allowed.

A veces nos sucedía que la línea "\r\n\r\n" no aparecía en el mensaje HTTP por culpa del navegador que hacía un request del favicon.ico de la página, rompiendo el servidor. Decidimos rechazar estas solicitudes requiriendo que el mensaje contenga "\r\n\r\n" obligatoriamente.

Inicialmente, nuestro servidor leía solo los primeros 4096 bytes de los requests y solo seguía leyendo cuando se trataba de un POST (para guardar el archivo completo). Sin embargo, al enviar archivos grandes nuestro servidor se colgaba porque a veces no llegaba a leer la totalidad de los headers. Es por eso que decidimos leer el mensaje completo (usando bytes ya que usar strings también nos llevaba a errores), y en el caso del POST, comprobamos nuevamente si se había leído el mensaje completo usando el header content-length.

#### *Análisis de protocolos y headers HTTP (Wireshark)*

En el siguiente documento, se puede observar la captura de los paquetes del protocolo HTTP usando Wireshark:  analisis\_HTTP\_wireshark.pdf .

En ambos casos (carga y descarga) podemos ver que se envían 2 mensajes (ignorando las solicitudes del favicon que rechazamos) y que se obtienen 2 respuestas. En el caso de la descarga, se realiza un GET tanto para solicitar el archivo HTML de la página inicial como para descargar el archivo que se solicitó. Las respuestas, incluyen en el payload los archivos solicitados para ambos casos. Por otro lado, en el caso de la carga, se realiza un GET al igual que en la descarga para la página inicial pero luego se realiza un POST para enviar los datos del archivo subido al servidor. Las respuestas incluyen payload solo cuando se solicita la página inicial.

Para los 3 mensajes bajo el método GET, los headers incluyen los campos Host, Connection, Upgrade-Insecure-Requests, User-Agent, Accept, Accept-Encoding y Accept-Language. Para el mensaje bajo el método POST, los headers fueron los mismos, pero incluyendo Content-Length, Cache-Control, Origin, Content-Type y Referer. Todas las respuestas incluyeron los campos Content-Type, Content-Length y Connection.

### Implementación de compresión gzip

Gzip es un algoritmo de compresión que trabaja sobre secuencias de bytes y reduce el tamaño de un archivo. Integrarlo como opción fue bastante sencillo. Primero, incluimos la opción de usar gzip desde la consola al iniciar el servidor. Segundo, mediante el header HTTP "Accept-Encoding" (previamente extraído del mensaje), veíamos si el cliente acepta la codificación. Finalmente, modificamos `manejar_descarga()` acordeamente.

### Experimentación

Para realizar la experimentación, iniciamos el servidor web en modo descarga pasando como parámetro los distintos archivos. Luego, nos conectamos desde otro dispositivo y tomamos una muestra de 10 descargas. Capturamos los paquetes de cada una de esas descargas usando Wireshark y medimos el tiempo de transferencia como el tiempo de llegada de la respuesta menos el tiempo de llegada del request. Una vez recolectados y clasificados los datos, armamos un dataset, generamos un boxplot en R y comparamos los tiempos de transferencia de cada archivo con y sin gzip.

### Imágenes

(Aclaremos que durante esta parte la experimentación sí se usó una máquina virtual)

Para esta parte del trabajo, usaremos el formato .jpg que ya aplica compresión con pérdida a las imágenes. Por lo tanto, la compresión en gzip no disminuirá significativamente el tamaño de los archivos enviados y se espera que los tiempos de transferencia con y sin compresión no sean tan distintos. Utilizamos las siguientes imágenes pequeña, mediana y grande:

Pequeña  
(18074 B = 17,6 KB)



Mediana  
(1148859 B = 1,09 MB)



Grande  
(7099027 B = 6,76 MB)

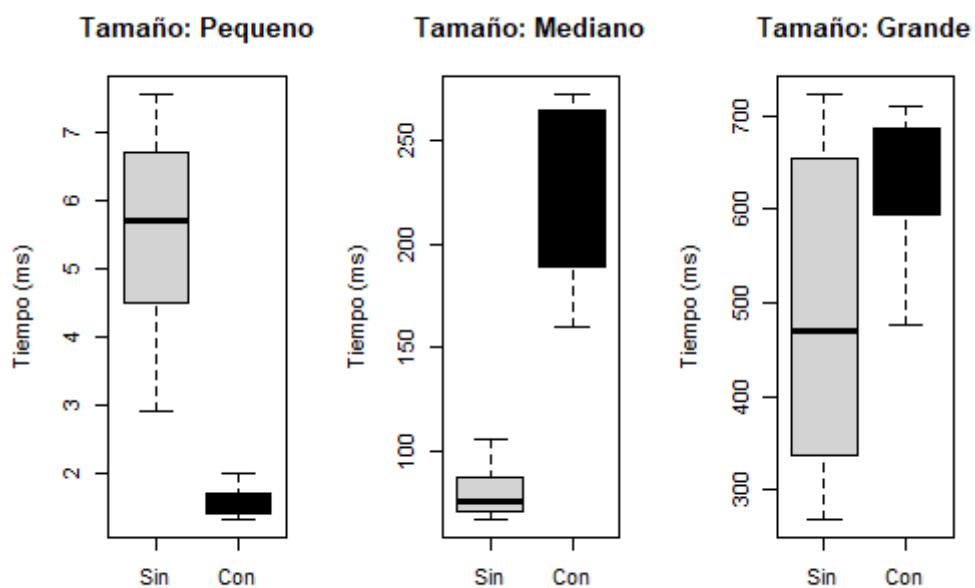


### Tiempo de transferencia (medidos en milisegundos):

| Pequeño  |          | Mediano  |          | Grande   |          |
|----------|----------|----------|----------|----------|----------|
| Con gzip | Sin gzip | Con gzip | Sin gzip | Con gzip | Sin gzip |
| 1,733    | 6,705    | 160,791  | 87,053   | 687,115  | 372,617  |
| 1,424    | 7,546    | 202,331  | 92,201   | 475,571  | 465,165  |
| 1,386    | 5,383    | 160,150  | 75,897   | 617,846  | 571,447  |
| 1,699    | 5,507    | 196,104  | 74,635   | 571,727  | 679,921  |

|       |        |         |         |         |         |
|-------|--------|---------|---------|---------|---------|
| 1,637 | 6,227  | 264,940 | 105,671 | 691,701 | 653,704 |
| 1,471 | 7,490  | 189,252 | 72,551  | 643,966 | 723,189 |
| 1,523 | 5,076  | 272,363 | 79,894  | 594,754 | 475,438 |
| 1,312 | 13,290 | 266,751 | 66,352  | 637,602 | 337,367 |
| 1,501 | 4,803  | 217,567 | 70,318  | 710,261 | 310,867 |
| 1,992 | 5,862  | 263,740 | 68,536  | 619,489 | 267,220 |

Gráficos y análisis:



Podemos ver que cuando el archivo es pequeño, los tiempos de transferencia son mucho menores cuando el archivo se comprime. Sin embargo, a medida que aumentamos el tamaño, las transferencias sin compresión toman menor tiempo. Por lo tanto, concluimos que cuando el contenido a comprimir es mayor, el tiempo que demora la compresión termina afectando al tiempo de transferencia ya que la disminución del tamaño del archivo no compensa el tiempo invertido en la compresión.

| <u>Resumen</u>        |                         | Pequeño      | Mediano      | Grande       |
|-----------------------|-------------------------|--------------|--------------|--------------|
| Tamaño original (B)   |                         | 18074        | 1148859      | 7099027      |
| Tamaño comprimido (B) |                         | 12213        | 1137749      | 7098917      |
| Ratio de compresión   | original/<br>comprimido | 1,4798984688 | 1,0097648954 | 1,0000154953 |
|                       | %                       | -47,98984688 | -0,97648954  | -0,00154953  |

|                                    |                       |               |                |               |
|------------------------------------|-----------------------|---------------|----------------|---------------|
| Tiempo de transferencia medio (ms) | s/compresión          | 5,5219        | 79,3158        | 485,6935      |
|                                    | c/compresión          | 1,5678        | 219,3989       | 625,0032      |
|                                    | <b>Diferencia (%)</b> | <b>-71,61</b> | <b>+176,61</b> | <b>+28,68</b> |

### Documentos

(Aclaremos que durante esta parte de la experimentación **no** se usó una máquina virtual)

Por otra parte, vamos a seguir experimentando con distintos libros (en formato .html para mayores posibilidades de compresión) obtenidos del [Proyecto Gutenberg](#): “The Raven” de Edgar Allan Poe (pequeño), “Dracula” de Bram Stoker (mediano) y “The Complete Works of William Shakespeare” de William Shakespeare (grande).

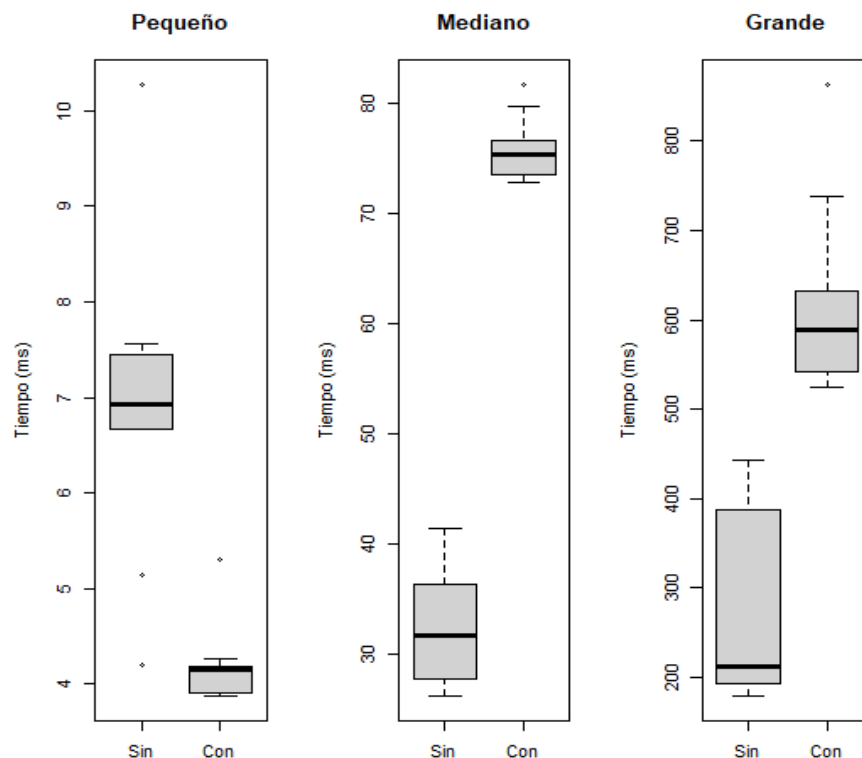
### Tiempo de transferencia (medidos en milisegundos):

| Pequeño  |          | Mediano  |          | Grande   |          |
|----------|----------|----------|----------|----------|----------|
| Con gzip | Sin gzip | Con gzip | Sin gzip | Con gzip | Sin gzip |
| 157,774* | 334,068* | 618,254* | 36,326   | 862,935  | 387,6    |
| 4,148    | 6,673    | 76,601   | 33,508   | 630,33   | 178,547  |
| 4,188    | 4,191    | 73,425   | 27,717   | 620,255  | 211,792  |
| 3,91     | 5,142    | 74,752   | 27,105   | 557,313  | 212,82   |
| 3,872    | 7,442    | 75,344   | 31,25    | 549,488  | 420,8    |
| 3,887    | 10,269   | 72,846   | 29,436   | 523,68   | 178,629  |
| 4,164    | 6,934    | 73,604   | 26,16    | 738,715  | 248,412  |
| 5,305    | 6,877    | 81,723   | 41,438   | 632,649  | 204,605  |
| 3,907    | 7,561    | 79,804   | 32,203   | 542,28   | 192,803  |
| 4,256    | 7,384    | 76,616   | 39,688   | 531,303  | 442,21   |

\* outliers de gran magnitud, excluidos de los respectivos gráficos.

### Gráficos y análisis:

La compresión reduce considerablemente el tiempo de transferencia para el caso del documento pequeño. Sin embargo, este se ve incrementado bastante para los casos de tamaño mediano y grande. Atribuimos esto a que el decremento del tiempo de transporte del servidor al cliente no logra compensar el tiempo de cómputo usado para reducir el tamaño del archivo.



| <u>Resumen</u>                     |                         | Pequeño         | Mediano         | Grande          |
|------------------------------------|-------------------------|-----------------|-----------------|-----------------|
| Tamaño original (KB)               |                         | 57,585          | 942,997         | 8.013,419       |
| Tamaño comprimido (KB)             |                         | 16,736          | 348,001         | 2.779,72        |
| Ratio de compresión                | original/<br>comprimido | 3,4407863289    | 2,7097537076    | 2,8828151756    |
|                                    | %                       | -244,0786328872 | -170,9753707604 | -188,2815175629 |
| Tiempo de transferencia medio (ms) | s/compresión            | 6,934           | 31,7265         | 212,306         |
|                                    | c/compresión            | 4,148           | 75,344          | 588,784         |
|                                    | <b>Diferencia (%)</b>   | <b>-40,28</b>   | <b>+137,48</b>  | <b>+177,33</b>  |

## Seguridad

El servidor presenta muchas vulnerabilidades importantes ya que utiliza HTTP sin cifrado, no tiene validaciones de entrada y maneja conexiones de forma secuencial. Dado que Internet fue diseñado para ser abierto y no seguro, estas debilidades permiten que Trudy pueda comprometer la confidencialidad, integridad y disponibilidad del servicio. Destacamos algunas de las posibles maneras en las que podría lograr sus objetivos.

*Intercepción del tráfico (Sniffing):*

Este tipo de ataque afecta a la confidencialidad. El servidor utiliza HTTP sin TLS (HTTPS es la extensión de HTTP que encripta usando TLS), por lo que todos los datos viajan en texto plano, incluyendo archivos enviados por POST y contenido solicitado por GET. Trudy puede capturar el tráfico con herramientas como Wireshark (al igual que hicimos nosotros) y leer archivos, nombres de usuario, contraseñas o cualquier información transmitida. Esta amenaza se origina en la ausencia total de cifrado. Para solucionarlo, se debería reemplazar el socket por uno seguro usando TLS, de modo que el servidor funcione sobre HTTPS y así el tráfico sea ilegible para atacantes.

#### *Modificación y corrupción de datos:*

Este problema afecta a la integridad del servidor. Durante la subida de archivos, el servidor no valida el nombre ni el contenido del archivo, permitiendo que Trudy envíe nombres con los cuales puede navegar por los distintos directorios de la computadora donde se ejecuta el servidor y sobrescriba archivos legítimos. Esta vulnerabilidad surge de la falta de controles sobre el input del usuario. Podría solucionarse sanitizando el nombre del archivo y asegurándose de que es válido, verificando el tamaño y tipo del archivo, y rechazando requests con headers inconsistentes.

#### *Ataque de denegación de servicio (DoS):*

Este ataque hace que el servidor deje de estar disponible. El servidor atiende un cliente por vez y no posee límites de tamaño ni timeouts en las conexiones. Trudy puede enviar un POST extremadamente grande o transmitir datos muy lentamente, haciendo que el servidor quede bloqueado y nadie más pueda utilizarlo. Esto explota la falta de límites en la lectura del body y la ausencia de tiempos máximos de espera. Se debería imponer un tamaño máximo permitido para uploads, cerrar conexiones lentas, y evitar que una única conexión bloquee el servicio.

#### *Extra: (Intento de) Implementación de un método básico de seguridad*

Decidimos seguir el ejemplo propuesto en la consigna y construir un método de autenticación mediante contraseña. Lo primero fue modificar el archivo HTML generado para incluir un campo de input de contraseña para el cliente (con ayuda de IA porque no dominamos el front-end). La idea es que el cliente envíe la contraseña como parte del path del GET o POST (que tampoco es muy seguro, pero cumple) y el servidor, por su parte, lo separe y convalide. Por ejemplo, en el caso de GET /download?pass=1234 HTTP/1.1, el servidor extrae la contraseña "1234", luego ve si es la correcta y decide. En caso de rechazar la solicitud (porque es incorrecta la contraseña), la respuesta será un 403 Forbidden, caso contrario continúa con la solicitud.

Luego, modificamos la función `server_start()` teniendo esto en cuenta para justamente los dos modos de arranque del servidor. La contraseña elegida fue `td2025`.

El archivo con las modificaciones es aquel titulado `server_fileTransfer_contraseña.py`.

## Conclusión

A lo largo del trabajo fuimos conociendo la parte práctica de aplicaciones que quizás conocíamos teóricamente por la cursada de la materia, pero desconocíamos cómo funcionaban en la vida real. Esto, sumado a otras cuestiones aprendidas, hacen que aumente el valor de la materia. Los principales aprendizajes obtenidos fueron la manera en que se aplican los protocolos en la interacción entre dispositivos, las nociones de cómo funciona y se estructura un servidor web (lo cuál usamos a diario e

ignoramos lo que sucede detrás), el manejo de sockets y mensajes HTTP en Python, entre otros. También, desconocíamos cómo funcionaba la carga y descarga de archivos en Internet, por lo que el trabajo nos ayudó a entender esto de manera más clara.

Encontramos muchos desafíos técnicos en la implementación, por lo que pasamos tiempo debugando e intentando buscar qué era lo que estaba fallando. Esto nos llevó a reestructurar el código del servidor muchas veces hasta encontrar una forma que nos convenciera y funcione de mejor manera. Hubo posibles mejoras que detectamos pero no implementamos ya que aumentaban la complejidad del código y el trabajo, implicando mayor tiempo y un proyecto más grande. Por ejemplo: recibir más de una conexión a la vez, comprobación de los nombres (para evitar colisiones) y contenido de los archivos subidos, límites de tamaño de subida, envío de los archivos por chunks (en vez de leerlo todo en memoria), y el manejo de favicon (que ignoramos).